

RA3D Software\RA3D Software.py

```

1  '''This is the main RA3D software. Run this file to use the program.
2  NOTE This program can only be run on a Raspberry Pi.
3  This program uses ArmController.py, SerialController.py, PrintController.py, and
4  TemperatureController.py
5  This program sets up the controllers and contains the code for all the GUI'''
6
7  from tkinter import *
8  from tkinter import messagebox
9  import tkinter.ttk as ttk
10 import threading
11 import time
12 from datetime import datetime
13 import RPi.GPIO as GPIO
14 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
15 from matplotlib.figure import Figure
16 import numpy as np
17
18 from SerialController import SerialController
19 from ArmController import ArmController, MoveParameters
20 from PrintController import PrintController
21 from TemperatureController import TemperatureController
22
23 class TkWindow(Tk):
24     # region init
25     def __init__(self):
26         Tk.__init__(self)
27         #All Settings
28         #Maps each display setting to a variable
29         #Simply add to this dictionary to add a setting
30         # "Display Name":("VariableName","Object")
31         self.settingsDict = {
32             "Default Print speed (mm/s)": ("speed","PC"), #print controller
33             "Acceleration": ("acceleration","PC"),
34             "Deceleration": ("decceleration","PC"),
35             "Ramp": ("ramp","PC"),
36             "Default Plate Height" : ("plateHeight","PC"),
37             "Ignore Flags" : ("ignoreFlags","PC"),
38             "deg/mm": ("extruder_deg_per_mm","AC"),
39             "Sync Print Parameters": ("syncWithPrintParameters","AC"), #set the default arm
40             "Debug Mode": ("DebugMode","self"),
41             "Print Debug Mode": ("PrintDebugMode", "self"),
42             #"Current Gcode Feedrate" : ("feedRate", "PC"),
43             "Printing Timeout Extra": ("timeoutExtra","PC"),
44             "Hard code printer speed to default": ("hardCodePrinterSpeed","PC"),
45             "Heated Filament Multiplier": ("heatedFilamentMultiplier","AC"),
46             "Coolend Mode": ("coolendMode","self"),

```

```

47         #"Drop Height": ("dropHeight","PC"),
48         "Default Angle": ("defaultAngle","PC") #Default pitch angle while printing
49         #"J5 Backlash Fix" : ("J5BacklashFix","custom")#Used to adjust the teensy fix,
not currently implemented on teensy
50     }
51     self.DebugMode = False #Will display important debug prints but not all of them
52     self.PrintDebugMode = False #Will display gcode lines and print coordinates
53     #Debug modes off will NOT hide errors
54
55     self.coolendMode = False #use for testing the arm and extrusion without requiring the
hotend to be at temperature
56
57     #Most cases for the blinking the LED should be shown by displaying a warning print
58     self.BlinkLED = False #Blink the LED when there is a problem
59     self.LEDOn = False #LED stays On, signifies in progress, overridden by blinkLED
60     self.ledThreadRunning = True # Used to stop the LED thread
61     self.unstoppableThreads = ["Serial Read Thread", "Serial Sort Thread","LED
Thread","MainThread", "Thread-1"] #Threads that will not be stopped on reset
62     self.LEDPin = 36
63     # Initialize the GPIO
64     GPIO.setmode(GPIO.BOARD)
65     # Initialize the LED
66     GPIO.setup(self.LEDPin, GPIO.OUT)
67
68     # Colors
69     # Taken from https://brand.uccs.edu/visual-guidelines/color on April 11th, 2026
70     self.colorCUBlack = "#000000"
71     self.colorCUGold = "#cfb87c"
72     self.colorDarkGray = "#565a5c"
73     self.colorLightGray = "#a2a4a3"
74     self.colorWhite = "#ffffff"
75
76     # Variables used to generalize colorations across the software
77     self.colorBG = self.colorCUBlack    # Background color
78     self.colorBG2 = self.colorDarkGray  # Background 2 color (used for backgrounds that
should stick out more)
79     self.colorBG3 = self.colorLightGray # Background 3 color (used for even brighter
backgrounds)
80     self.colorFG = self.colorWhite      # Foreground color (like text)
81     self.colorAccent = self.colorCUGold # Accent colors
82
83     # Style configurations
84     s = ttk.Style()
85     s.theme_use('default')
86     s.configure("TProgressbar",
87                 thickness = 50,                # Height of the bar
88                 background = self.colorAccent, # Color of the bar that is filled in
89                 troughcolor = self.colorBG3,    # Color of the bar that isn't filled in
90                 pbarrelief = SOLID)
91     s.configure("TSeparator", background=self.colorBG2)
92     # s.configure("TCombobox",

```

```

93         #         foreground=self.colorFG,
94         #         background=self.colorBG2,
95         #         selectbackground=self.colorBG2,
96         #         fieldbackground=self.colorBG2)
97
98     self.root = self
99     # Set the window title
100    self.title("RA3D Control Software")
101    # (TEMP) Set window as topmost
102    # self.attributes('-topmost', True)
103    self.updateDelay = 150 # Delay between update function calls in milliseconds
104    # Set the window dimensions and position on screen
105    w = 1220 # Window width
106    h = 685 # Window height
107    ws = self.winfo_screenwidth() # Get screen width
108    hs = self.winfo_screenheight() # Get screen height
109    x = int((ws/2) - (w/2)) # Calculate x position for window to be in the center of the
screen
110    y = int((hs/2) - (h/2)) # Calculate y position for window to be in the center of the
screen
111    self.geometry(f"{w}x{h}+{x}+{y}") # Set the width, height, x, and y values
112    self.root.config(bg=self.colorBG)
113
114    # Instantiate objects for the various controller classes
115    self.serialController = SerialController(self.root)
116    self.armController = ArmController(self.root, self.serialController)
117    self.printController = PrintController(self.root, self.armController)
118    self.temperatureController = TemperatureController(self.root)
119
120    # Create and draw widgets onto the window
121    self.createTabs()
122    if self.DebugMode:
123        self.root.terminalPrint("GUI created")
124
125    self.serialController.refreshCOMPorts()
126
127
128    self.root.protocol("WM_DELETE_WINDOW", self.shutdownProgram)
129
130    self.printThreadStarted = False
131    #Set origin last so everything is in place
132    self.armController.setOrigin(origin=self.root.printController.recommendedOrigin)
133
134    self.ledThread = threading.Thread(target=self.updateLED, daemon=True, name="LED
Thread")
135    self.ledThread.start()
136    self.serialController.autoConnect()
137    self.root.bind("<Button-1>", self.clearEntryFocus)
138    # Set up a call to the update function after updateDelay milliseconds
139    self.update()

```

```
140         self.printController.autoSelectFile("../Programming Experiments/Gcode
Files/one_layer.gcode") #for testing purposes, can change to None to not auto select
141
142     #endregion init
143
144     #region Shutdown
145     # This function is meant to do various shutdown tasks so the program doesn't break
anything
146     def shutdownProgram(self):
147         try:
148             # Move the arm to a safe position before shutting down, will not move safe if not
calibrated
149             # This is necessary because if the arm is not calibrated
150             if not self.armController.calibrationOverridden:
151                 self.armController.moveSafe()
152                 self.ledThreadRunning = False
153                 # Release the GPIO pins from use
154                 GPIO.cleanup()
155                 # Close the window
156                 self.root.destroy()
157         except:
158             #if failed then destroy the window anyway, we don't want it to be stuck
159             self.root.destroy()
160     #endregion
161
162     #region Tabs
163     # Creates the interface tabs
164     def createTabs(self):
165         # Create a notebook to manage the tabs
166         self.notebook = ttk.Notebook(self.root)
167         self.notebook.pack(fill="both", expand=True)
168         # Create the tabs
169         self.homeTab = Frame(self.notebook, bg=self.colorBG)
170         self.armTab = Frame(self.notebook, bg=self.colorBG)
171         self.toolTab = Frame(self.notebook, bg=self.colorBG)
172         self.debugTab = Frame(self.notebook, bg=self.colorBG)
173         self.settingsTab = Frame(self.notebook, bg=self.colorBG)
174         # Put the tabs on screen
175         self.homeTab.pack(fill="both", expand=True)
176         self.armTab.pack(fill="both", expand=True)
177         self.toolTab.pack(fill="both", expand=True)
178         self.debugTab.pack(fill="both", expand=True)
179         self.settingsTab.pack(fill="both", expand=True)
180         # Add the tabs to the notebook
181         self.notebook.add(self.homeTab, text="Home")
182         self.notebook.add(self.armTab, text="Arm Control")
183         self.notebook.add(self.toolTab, text="Extruder")
184         self.notebook.add(self.debugTab, text="Debug")
185         self.notebook.add(self.settingsTab, text="Settings")
186         # Call the various functions for creating the widgets in each tab
187         self.fillHomeTab()
```



```

188         self.fillArmTab()
189         self.fillToolTab()
190         self.fillDebugTab()
191         self.fillSettingsTab()
192
193         # Create a status label for immediate user response
194         self.statusBarFrame = Frame(self.root, height=20, bg=self.colorAccent)
195         self.statusBarFrame.pack(fill="both", expand=True, side="bottom")
196         self.statusLabel = Label(self.statusBarFrame, text="Status: Example",
bg=self.colorAccent, fg=self.colorBG)
197         self.statusLabel.grid(row=0, column=0, sticky=W+N+S)
198
199         def fillHomeTab(self):
200             # =====| Print Info Frame |=====
201             printInfoWidth = 600
202             self.printInfoHomeFrame = Frame(self.homeTab, bg=self.colorBG2, width=printInfoWidth,
height=350)
203             self.printInfoHomeFrame.grid(row=0, column=0, padx=5, pady=5, sticky=W+E+N+S)
204             # Print status label (Printing, paused, etc.)
205             self.printStatusHomeLabel = Label(self.printInfoHomeFrame, text="IDLE...", font=
("TkDefaultFont", 30, "bold"), bg=self.colorBG2, fg=self.colorFG)
206             self.printStatusHomeLabel.grid(row=0, column=0, columnspan=3, rowspan=2, padx=5,
pady=5, sticky=N+W)
207             self.printStatusHomeLabel.bind("<Button-1>", self.windowDimensions)
208             # Start button
209             self.startPrintHomeButton = Button(self.printInfoHomeFrame, text="Start", width=10,
height=2, command=self.printController.startPrint, state="disabled", bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
210             self.startPrintHomeButton.grid(row=0, column=3, padx=5, pady=5, sticky=E)
211             # Pause button
212             self.pausePrintHomeButton = Button(self.printInfoHomeFrame, text="Pause", width=10,
height=2, command=self.printController.pausePrint, state="disabled", bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
213             self.pausePrintHomeButton.grid(row=1, column=3, padx=5, pady=5, sticky=E)
214             # Stop button
215             self.stopPrintHomeButton = Button(self.printInfoHomeFrame, text="Stop", width=10,
height=2, command=self.printController.cancelPrint, state="disabled", bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
216             self.stopPrintHomeButton.grid(row=2, column=3, padx=5, pady=5, sticky=E)
217             # Select file button
218             self.selectFileHomeButton = Button(self.printInfoHomeFrame, text="Select File",
width=10, command=self.printController.selectFile, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
219             self.selectFileHomeButton.grid(row=4, column=0, padx=5, pady=5, sticky=W)
220             # File name label
221             self.selectedFileHomeLabel = Label(self.printInfoHomeFrame, text="Please select a
file", bg=self.colorBG2, fg=self.colorFG)
222             self.selectedFileHomeLabel.grid(row=4, column=1, padx=5, pady=5, sticky=W)
223             # Progress label
224             self.progressHomeLabel = Label(self.printInfoHomeFrame, text="0%", bg=self.colorBG2,
fg=self.colorFG)
225             self.progressHomeLabel.grid(row=2, column=0, padx=5, pady=5, sticky=W+S)

```

```

226         # Progress bar
227         self.printProgressBarHome = ttk.Progressbar(self.printInfoHomeFrame,
orient=HORIZONTAL, length=printInfoWidth - 20, mode="determinate")
228         self.printProgressBarHome.grid(row=3, column=0, columnspan=4, padx=10, pady=5,
sticky=W+E)
229         self.printProgressBarHome['value'] = 50
230
231         self.currentJ7 = Label(self.printInfoHomeFrame, text="Extruded: 0 mm",
bg=self.colorBG2, fg=self.colorFG)
232         self.currentJ7.grid(row=6, column=0, padx=5, pady=5, sticky=N+S)
233
234         # =====| Thermals Frame |=====
235         self.thermalsHomeFrame = Frame(self.homeTab, width=printInfoWidth, height=350,
bg=self.colorBG2)
236         self.thermalsHomeFrame.grid(row=0, column=1, padx=5, pady=5, sticky=W+E+N+S)
237         self.thermalsHomeLabel = Label(self.thermalsHomeFrame, text="Thermals", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
238         self.thermalsHomeLabel.grid(row=0, column=0, columnspan=2, sticky=W+N+S, padx=5,
pady=5)
239         ttk.Separator(self.thermalsHomeFrame, orient='horizontal').grid(row=1, column=0,
columnspan=6, sticky=W+E)
240         self.thermalNameLabel = Label(self.thermalsHomeFrame, text="Name", bg=self.colorBG2,
fg=self.colorFG)
241         self.thermalNameLabel.grid(row=2, column=0, columnspan=2, sticky=W, padx=5, pady=5)
242         self.thermalActualLabel = Label(self.thermalsHomeFrame, text="Actual (°C)",
bg=self.colorBG2, fg=self.colorFG)
243         self.thermalActualLabel.grid(row=2, column=2, sticky=E, padx=5, pady=5)
244         self.thermalTargetLabel = Label(self.thermalsHomeFrame, text="Target (°C)",
bg=self.colorBG2, fg=self.colorFG)
245         self.thermalTargetLabel.grid(row=2, column=3, sticky=E, padx=5, pady=5)
246         self.thermalEnabledLabel = Label(self.thermalsHomeFrame, text="Enabled",
bg=self.colorBG2, fg=self.colorFG)
247         self.thermalEnabledLabel.grid(row=2, column=4, sticky=E, padx=5, pady=5)
248         ttk.Separator(self.thermalsHomeFrame, orient='horizontal').grid(row=3, column=0,
columnspan=6, sticky=W+E)
249         # Hotend temp info
250         self.hotendHomeLabel = Label(self.thermalsHomeFrame, text="Hotend", bg=self.colorBG2,
fg=self.colorFG)
251         self.hotendHomeLabel.grid(row=4, column=0, sticky=W, padx=5, pady=5)
252         self.hotendHomeActual = Label(self.thermalsHomeFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG)
253         self.hotendHomeActual.grid(row=4, column=2, sticky=E, padx=5, pady=5)
254         #self.hotendHomeTarget = Label(self.thermalsHomeFrame, text="xxx")
255         self.hotendHomeTarget = Entry(self.thermalsHomeFrame, width=5, justify="right",
bg=self.colorBG2, fg=self.colorFG)
256         self.hotendHomeTarget.insert(0, "0")
257         self.hotendHomeTarget.grid(row=4, column=3, sticky=E, padx=5, pady=5)
258         self.hotendHomeEnableButton = Button(self.thermalsHomeFrame, text="OFF", width=4,
command=self.temperatureController.enableHotendControl, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
259         self.hotendHomeEnableButton.grid(row=4, column=4, sticky=E, padx=5, pady=5)

```

```

260         self.hotendHomeLegend = Label(self.thermalsHomeFrame, text=" ", bg="#FF0000",
width=2)
261         self.hotendHomeLegend.grid(row=4, column=5, padx=5, pady=5, sticky=W)
262
263         # Bed temp info
264         self.bedHomeLabel = Label(self.thermalsHomeFrame, text="Bed", bg=self.colorBG2,
fg=self.colorFG)
265         self.bedHomeLabel.grid(row=5, column=0, sticky=W, padx=5, pady=5)
266         self.bedHomeActual = Label(self.thermalsHomeFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG)
267         self.bedHomeActual.grid(row=5, column=2, sticky=E, padx=5, pady=5)
268         #self.bedHomeTarget = Label(self.thermalsHomeFrame, text="xxx")
269         self.bedHomeTarget = Entry(self.thermalsHomeFrame, width=5, justify="right",
bg=self.colorBG2, fg=self.colorFG)
270         self.bedHomeTarget.insert(0, "0")
271         self.bedHomeTarget.grid(row=5, column=3, sticky=E, padx=5, pady=5)
272         self.bedHomeEnableButton = Button(self.thermalsHomeFrame, text="OFF", width=4,
command=self.temperatureController.enableBedControl, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
273         self.bedHomeEnableButton.grid(row=5, column=4, sticky=E, padx=5, pady=5)
274         self.bedHomeLegend = Label(self.thermalsHomeFrame, text=" ", bg="#0000FF", width=2)
275         self.bedHomeLegend.grid(row=5, column=5, padx=5, pady=5, sticky=W)
276
277         # Live temperatures plot
278         ttk.Separator(self.thermalsHomeFrame, orient='horizontal').grid(row=6, column=0,
columnspan=6, sticky=W+E)
279         self.thermalFig = Figure(figsize=(5.75, 2.5), dpi=100)
280         self.thermalPlot = self.thermalFig.add_subplot(111)
281         self.thermalHotendLine, = self.thermalPlot.plot([], [], 'r-', label="Hotend")
282         self.thermalBedLine, = self.thermalPlot.plot([], [], 'b-', label="Bed")
283         self.thermalCanvas = FigureCanvasTkAgg(self.thermalFig,
master=self.thermalsHomeFrame)
284         self.thermalCanvas.get_tk_widget().grid(row=7, column=0, columnspan=6,
sticky=W+E+N+S, padx=5, pady=5)
285         self.thermalPlot.set_ylabel("Temperature (C)")
286         self.thermalPlot.set_xlabel("Time (s)")
287         self.thermalPlot.margins(x=0.1, y=0.05)
288         self.thermalPlot.spines['right'].set_visible(False)
289         self.thermalPlot.spines['top'].set_visible(False)
290         # Lots of color setting
291         self.thermalFig.set_facecolor(self.thermalsHomeFrame.cget("bg"))
292         self.thermalPlot.spines["left"].set_color(self.colorFG)
293         self.thermalPlot.spines["bottom"].set_color(self.colorFG)
294         self.thermalPlot.set_facecolor(self.colorBG3)
295         self.thermalPlot.tick_params(axis='x', colors=self.colorFG)
296         self.thermalPlot.tick_params(axis='y', colors=self.colorFG)
297         self.thermalPlot.xaxis.label.set_color(self.colorFG)
298         self.thermalPlot.yaxis.label.set_color(self.colorFG)
299         self.thermalPlot.grid(True) # Add a grid
300         self.hotendTargetLine = self.thermalPlot.axhline(y=self.temperatureControlle-
r.hotendTargetTemp, color='r', linestyle='--')

```

```

301         self.bedTargetLine = self.thermalPlot.axhline(y=self.temperatureControlle-
r.bedTargetTemp, color='b', linestyle='--')
302         self.yMax = 100 # Used for y-axis scaling of the plot
303         self.thermalCanvas.get_tk_widget().bind("<Button-1>", self.clearEntryFocus)
304
305         # =====| Arm Info Frame |=====
306         self.armInfoHomeFrame = Frame(self.homeTab, width=600, height=200, bg=self.colorBG2)
307         self.armInfoHomeFrame.grid(row=1, column=0, padx=5, pady=5, sticky=W+E+N+S)
308         Label(self.armInfoHomeFrame, text="Arm Info", font=("TkDefaultFont", 12, "bold"),
bg=self.colorBG2, fg=self.colorFG).grid(row=0, column=0, columnspan=2, sticky=W+N+S, padx=5,
pady=5)
309         ttk.Separator(self.armInfoHomeFrame, orient='horizontal').grid(row=1, column=0,
columnspan=4, sticky=W+E)
310         self.connectedStatusHome = Label(self.armInfoHomeFrame, text="Arm Disconnected",
width=30, justify=LEFT, anchor=W, bg=self.colorBG2)
311         self.connectedStatusHome.grid(row=2, column=0, padx=5, pady=5, sticky=W)
312         self.calibrationStatusHome = Label(self.armInfoHomeFrame, text="Arm NOT Calibrated",
width=30, justify=LEFT, anchor=W, bg=self.colorBG2)
313         self.calibrationStatusHome.grid(row=3, column=0, rowspan=2, padx=5, pady=5,
sticky=W)
314         self.calibrationHomeButton = Button(self.armInfoHomeFrame, text="Calibrate",
command=self.armController.startArmCalibration, width=10, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
315         self.calibrationHomeButton.grid(row=5, column=0, padx=5, pady=5, sticky=W)
316
317         ttk.Separator(self.armInfoHomeFrame, orient='vertical').grid(row=2, column=1,
rowspan=5, sticky=N+S)
318         Label(self.armInfoHomeFrame, text="Bed Leveling", font=("TkDefaultFont", 12,
"bold"), bg=self.colorBG2, fg=self.colorFG).grid(row=2, column=2, columnspan=2,
sticky=W+N+E, padx=5, pady=5)
319         ttk.Separator(self.armInfoHomeFrame, orient='horizontal').grid(row=3, column=2,
columnspan=2, sticky=W+E)
320         self.startLevelHome = Button(self.armInfoHomeFrame, text="Start Level", width=15,
command=self.printController.startPrintBedCalibration, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
321         self.startLevelHome.grid(row=4, column=2, padx=5, pady=5)
322         self.nextLevelHome = Button(self.armInfoHomeFrame, text= "Next Corner", width=15,
command=self.printController.nextBedCalibration, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
323         self.nextLevelHome.grid(row=4, column=3, padx=5, pady=5)
324         self.sweepCornersHome = Button(self.armInfoHomeFrame, text= "Sweep Corners",
width=15, command=self.printController.startCornerSweep, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
325         self.sweepCornersHome.grid(row=5, column=2, padx=5, pady=5)
326         self.cornerLabelHome = Label(self.armInfoHomeFrame, text="Current corner: N/A",
bg=self.colorBG2, fg=self.colorFG)
327         self.cornerLabelHome.grid(row=5, column=3, padx=5, pady=5)
328         self.sweepCornersFullHome = Button(self.armInfoHomeFrame, text= "Full Corner Sweep",
width=15, command=self.printController.startFullCornerSweep, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
329         self.sweepCornersFullHome.grid(row=6, column=2, padx=5, pady=5)
330         self.cancelAnyHome = Button(self.armInfoHomeFrame, text= "Cancel Any", width=15,
command=self.printController.cancelAny, bg=self.colorAccent, fg=self.colorBG, font=

```

```

("TkDefaultFont", 10, "bold"))
331         self.cancelAnyHome.grid(row=6, column=3, padx=5, pady=5)
332
333         # =====| Credits Frame |=====
334         self.creditsHomeFrame = Frame(self.homeTab, width=600, height=200, bg=self.colorBG2)
335         self.creditsHomeFrame.grid(row=1, column=1, padx=5, pady=5, sticky=W+E+N+S)
336         Label(self.creditsHomeFrame, text="RA3D", font=("TkDefaultFont", 20, "bold"),
bg=self.colorBG2, fg=self.colorFG).grid(row=0, column=0, sticky=EW)
337         CreditsText1 = "Designed and implemented by the RA3D Team (Robotic Arm 3D)"
338         CreditsText2 = "Team Members: Justin Fauson, Cody Blough, Jonathan Dinan, Mateo
Osorio, \n and Jonathan Pederson"
339         CreditsText3 = "Sponsor: Dr. Sezer Ozerinc"
340         Label(self.creditsHomeFrame, text=CreditsText1, bg=self.colorBG2,
fg=self.colorFG).grid(row=1, column=0, sticky=W, padx=5)
341         Label(self.creditsHomeFrame, text=CreditsText2, justify="left", bg=self.colorBG2,
fg=self.colorFG).grid(row=2, column=0, sticky=W, padx=5)
342         Label(self.creditsHomeFrame, text=CreditsText3, justify="left", bg=self.colorBG2,
fg=self.colorFG).grid(row=3, column=0, sticky=W, padx=5)
343
344         def fillArmTab(self):
345             # =====| Serial Frame |=====
346             self.serialFrame = Frame(self.armTab, bg=self.colorBG2)
347             self.serialFrame.grid(row=0, column=0, padx=5, pady=5, sticky=W+N+E+S)
348             # Create label for Serial Frame
349             self.serialLabel = Label(self.serialFrame, text="Serial Port", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
350             self.serialLabel.grid(row=0, column=0, columnspan=2, padx=5, pady=5, sticky=W)
351             # Refresh button
352             self.refreshCOMButton = Button(self.serialFrame, text="↻",
command=self.serialController.refreshCOMPorts, width=2, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
353             self.refreshCOMButton.grid(row=1, column=0, padx=5, pady=5)
354             # Create dropdown list of all serial COM ports
355             self.portList = [] # Start with blank list
356             self.portSelection = StringVar(value="Select Port") # Create a StringVar to hold the
current dropdown selection
357             self.portSelection.trace("w", self.portSelectionChanged) # Connect portSelection
changing to a function call to detect when the selected option changes
358             self.portDropdown = ttk.Combobox(self.serialFrame, width=10,
textvariable=self.portSelection, state="readonly") # Create the dropdown
359             self.portDropdown["values"] = self.portList
360             self.portDropdown.grid(row=1, column=1, padx=5, pady=5)
361
362             # Create button for connecting to port
363             self.connectButton = Button(self.serialFrame, text="Connect",
command=self.serialController.serialConnect, width=10, state="disabled",
bg=self.colorAccent, fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
364             self.connectButton.grid(row=1, column=2, padx=5, pady=5)
365             # Status label
366             self.portStatusLabel = Label(self.serialFrame, text="Status: Disconnected",
bg=self.colorBG2, fg=self.colorFG)
367             self.portStatusLabel.grid(row=2, column=0, columnspan=3, padx=5, pady=5, sticky=W)

```

```

368         # Reset button
369         self.resetButton = Button(self.serialFrame, text="Reset",
command=self.armController.reset, width=10, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
370         self.resetButton.grid(row=3, column=1, padx=5, pady=5)
371         # =====| Reported Position Frame |=====
372         self.reportedPosFrame = Frame(self.armTab, bg=self.colorBG2)
373         self.reportedPosFrame.grid(row=0, column=1, columnspan=2, padx=5, pady=5,
sticky=W+N+E+S)
374         # Reported Position label
375         self.reportedPosLabel = Label(self.reportedPosFrame, text="Reported Position:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
376         self.reportedPosLabel.grid(row=0, column=0, columnspan=5, padx=5, pady=5, sticky=W)
377         # Request position button
378         self.requestPosButton = Button(self.reportedPosFrame, text="Request Position",
command=self.armController.requestPositionManual, width=15, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
379         self.requestPosButton.grid(row=0, column=1, padx=5, pady=5, sticky=E)
380         # Reported position coordinate labels
381         # XYZ
382         self.xyzPosFrame = Frame(self.reportedPosFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
383         self.xyzPosFrame.grid(row=1, column=0, padx=5, pady=5)
384         # Create them
385         self.xCurCoordLabel = Label(self.xyzPosFrame, text="X:", bg=self.colorBG2,
fg=self.colorFG)
386         self.xCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
387         self.yCurCoordLabel = Label(self.xyzPosFrame, text="Y:", bg=self.colorBG2,
fg=self.colorFG)
388         self.yCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
389         self.zCurCoordLabel = Label(self.xyzPosFrame, text="Z:", bg=self.colorBG2,
fg=self.colorFG)
390         self.zCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
391         self.RxCurCoordLabel = Label(self.xyzPosFrame, text="Rx:", bg=self.colorBG2,
fg=self.colorFG)
392         self.RxCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
393         self.RyCurCoordLabel = Label(self.xyzPosFrame, text="Ry:", bg=self.colorBG2,
fg=self.colorFG)
394         self.RyCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
395         self.RzCurCoordLabel = Label(self.xyzPosFrame, text="Rz:", bg=self.colorBG2,
fg=self.colorFG)
396         self.RzCurCoord = Label(self.xyzPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
397         # Display them
398         self.xCurCoordLabel.grid(row=0, column=0, padx=5, pady=5)
399         self.xCurCoord.grid(row=0, column=1, padx=5, pady=5)
400         self.yCurCoordLabel.grid(row=0, column=2, padx=5, pady=5)

```

```

401     self.yCurCoord.grid(row=0, column=3, padx=5, pady=5)
402     self.zCurCoordLabel.grid(row=0, column=4, padx=5, pady=5)
403     self.zCurCoord.grid(row=0, column=5, padx=5, pady=5)
404     self.RxCurCoordLabel.grid(row=1, column=4, padx=5, pady=5)
405     self.RxCurCoord.grid(row=1, column=5, padx=5, pady=5)
406     self.RyCurCoordLabel.grid(row=1, column=2, padx=5, pady=5)
407     self.RyCurCoord.grid(row=1, column=3, padx=5, pady=5)
408     self.RzCurCoordLabel.grid(row=1, column=0, padx=5, pady=5)
409     self.RzCurCoord.grid(row=1, column=1, padx=5, pady=5)
410
411     # Angles
412     self.jointPosFrame = Frame(self.reportedPosFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
413     self.jointPosFrame.grid(row=1, column=1, padx=5, pady=5)
414     # Create them
415     self.J1CurCoordLabel = Label(self.jointPosFrame, text="J1:", bg=self.colorBG2,
fg=self.colorFG)
416     self.J1CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
417     self.J2CurCoordLabel = Label(self.jointPosFrame, text="J2:", bg=self.colorBG2,
fg=self.colorFG)
418     self.J2CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
419     self.J3CurCoordLabel = Label(self.jointPosFrame, text="J3:", bg=self.colorBG2,
fg=self.colorFG)
420     self.J3CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
421     self.J4CurCoordLabel = Label(self.jointPosFrame, text="J4:", bg=self.colorBG2,
fg=self.colorFG)
422     self.J4CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
423     self.J5CurCoordLabel = Label(self.jointPosFrame, text="J5:", bg=self.colorBG2,
fg=self.colorFG)
424     self.J5CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
425     self.J6CurCoordLabel = Label(self.jointPosFrame, text="J6:", bg=self.colorBG2,
fg=self.colorFG)
426     self.J6CurCoord      = Label(self.jointPosFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
427     # Display them
428     self.J1CurCoordLabel.grid(row=0, column=0, padx=5, pady=5)
429     self.J1CurCoord.grid(row=0, column=1, padx=5, pady=5)
430     self.J2CurCoordLabel.grid(row=0, column=2, padx=5, pady=5)
431     self.J2CurCoord.grid(row=0, column=3, padx=5, pady=5)
432     self.J3CurCoordLabel.grid(row=0, column=4, padx=5, pady=5)
433     self.J3CurCoord.grid(row=0, column=5, padx=5, pady=5)
434     self.J4CurCoordLabel.grid(row=1, column=0, padx=5, pady=5)
435     self.J4CurCoord.grid(row=1, column=1, padx=5, pady=5)
436     self.J5CurCoordLabel.grid(row=1, column=2, padx=5, pady=5)
437     self.J5CurCoord.grid(row=1, column=3, padx=5, pady=5)
438     self.J6CurCoordLabel.grid(row=1, column=4, padx=5, pady=5)
439     self.J6CurCoord.grid(row=1, column=5, padx=5, pady=5)

```

```

440
441     # =====| Calibration Frame |=====
442     self.calibrationFrame = Frame(self.armTab, bg=self.colorBG2)
443     self.calibrationFrame.grid(row=1, column=0, padx=5, pady=5)
444     # Calibration label
445     self.calibrationLabel = Label(self.calibrationFrame, text="Calibration:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
446     self.calibrationLabel.grid(row=0, column=0, padx=5, pady=5, sticky=W+N)
447     # Full calibration button
448     self.calibrateButton = Button(self.calibrationFrame, text="Calibrate",
command=self.armController.startArmCalibration, width=10, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
449     self.calibrateButton.grid(row=1, column=0, padx=5, pady=5, sticky=W+E)
450     # Calibration offsets
451     self.calOffsetFrame = Frame(self.calibrationFrame, bg=self.colorBG2)
452     self.calOffsetFrame.grid(row=2, column=0, padx=5, pady=5, sticky=W+E)
453     self.offsetLabel = Label(self.calOffsetFrame, text="Joint Offsets:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
454     self.offsetLabel.grid(row=0, column=0, columnspan=5, padx=5, pady=5, sticky=W)
455     # Make the widgets
456     self.J10ffsetLabel = Label(self.calOffsetFrame, text="J1:", bg=self.colorBG2,
fg=self.colorFG)
457     self.J10ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
458     self.J20ffsetLabel = Label(self.calOffsetFrame, text="J2:", bg=self.colorBG2,
fg=self.colorFG)
459     self.J20ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
460     self.J30ffsetLabel = Label(self.calOffsetFrame, text="J3:", bg=self.colorBG2,
fg=self.colorFG)
461     self.J30ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
462     self.J40ffsetLabel = Label(self.calOffsetFrame, text="J4:", bg=self.colorBG2,
fg=self.colorFG)
463     self.J40ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
464     self.J50ffsetLabel = Label(self.calOffsetFrame, text="J5:", bg=self.colorBG2,
fg=self.colorFG)
465     self.J50ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
466     self.J60ffsetLabel = Label(self.calOffsetFrame, text="J6:", bg=self.colorBG2,
fg=self.colorFG)
467     self.J60ffsetEntry = Entry(self.calOffsetFrame, width=4, bg=self.colorBG2,
fg=self.colorFG)
468
469     # Grid the widgets
470     self.J10ffsetLabel.grid(row=1, column=0, padx=5, pady=5)
471     self.J10ffsetEntry.grid(row=1, column=1, padx=5, pady=5)
472     self.J20ffsetLabel.grid(row=1, column=2, padx=5, pady=5)
473     self.J20ffsetEntry.grid(row=1, column=3, padx=5, pady=5)
474     self.J30ffsetLabel.grid(row=1, column=4, padx=5, pady=5)
475     self.J30ffsetEntry.grid(row=1, column=5, padx=5, pady=5)

```



```

476 self.J40offsetLabel.grid(row=2, column=0, padx=5, pady=5)
477 self.J40offsetEntry.grid(row=2, column=1, padx=5, pady=5)
478 self.J50offsetLabel.grid(row=2, column=2, padx=5, pady=5)
479 self.J50offsetEntry.grid(row=2, column=3, padx=5, pady=5)
480 self.J60offsetLabel.grid(row=2, column=4, padx=5, pady=5)
481 self.J60offsetEntry.grid(row=2, column=5, padx=5, pady=5)
482
483 # Auto fill a value of '0'
484 self.J10offsetEntry.insert(0, "0")
485 self.J20offsetEntry.insert(0, "0")
486 self.J30offsetEntry.insert(0, "0")
487 self.J40offsetEntry.insert(0, "0")
488 self.J50offsetEntry.insert(0, "0")
489 self.J60offsetEntry.insert(0, "0")
490
491 # Individual calibration buttons
492 self.indivCalFrame = Frame(self.calibrationFrame, bg=self.colorBG2)
493 self.indivCalFrame.grid(row=3, column=0, padx=5, pady=5, sticky=W+E)
494 # Label
495 self.indivCalLabel = Label(self.indivCalFrame, text="Individual Calibrations:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
496 self.indivCalLabel.grid(row=0, column=0, columnspan=3, padx=5, pady=5, sticky=W)
497 # Make the buttons
498 self.calJ1Button = Button(self.indivCalFrame, text="Cal J1", command=lambda:
self.armController.startSpecificCalibration(1, 0, 0, 0, 0, 0), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
499 self.calJ2Button = Button(self.indivCalFrame, text="Cal J2", command=lambda:
self.armController.startSpecificCalibration(0, 1, 0, 0, 0, 0), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
500 self.calJ3Button = Button(self.indivCalFrame, text="Cal J3", command=lambda:
self.armController.startSpecificCalibration(0, 0, 1, 0, 0, 0), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
501 self.calJ4Button = Button(self.indivCalFrame, text="Cal J4", command=lambda:
self.armController.startSpecificCalibration(0, 0, 0, 1, 0, 0), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
502 self.calJ5Button = Button(self.indivCalFrame, text="Cal J5", command=lambda:
self.armController.startSpecificCalibration(0, 0, 0, 0, 1, 0), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"), state="disabled")
503 self.calJ6Button = Button(self.indivCalFrame, text="Cal J6", command=lambda:
self.armController.startSpecificCalibration(0, 0, 0, 0, 0, 1), width=7, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
504 # Place buttons
505 self.calJ1Button.grid(row=1, column=0, padx=5, pady=5,)
506 self.calJ2Button.grid(row=1, column=1, padx=5, pady=5)
507 self.calJ3Button.grid(row=1, column=2, padx=5, pady=5)
508 self.calJ4Button.grid(row=2, column=0, padx=5, pady=5)
509 self.calJ5Button.grid(row=2, column=1, padx=5, pady=5)
510 self.calJ6Button.grid(row=2, column=2, padx=5, pady=5)
511 # ==== Open Post Calibration ====
512 self.postCalibrationButton = Button(self.calibrationFrame, text="Post Calibration",
command=self.createPostCalibration, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))

```

```

513     self.postCalibrationButton.grid(row=4,column=0, padx=5, pady=5)
514     # =====| Tests Frame |=====
515     self.armTestsFrame = Frame(self.armTab, bg=self.colorBG2)
516     self.armTestsFrame.grid(row=1, column=1, padx=5, pady=5, sticky=W+E+N+S)
517     # Label
518     self.armTestsLabel = Label(self.armTestsFrame, text="Arm Tests:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
519     self.armTestsLabel.grid(row=0, column=0, padx=5, pady=5, sticky=W+N)
520
521     # Limit switch test
522     self.limitTestFrame = Frame(self.armTestsFrame, bg=self.colorBG2)
523     self.limitTestFrame.grid(row=1, column=0, padx=5, pady=5, sticky=W)
524     self.limitTestButton = Button(self.limitTestFrame, text="Test Limit Switches",
command=self.armController.toggleLimitTest, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
525     self.limitTestButton.grid(row=0, column=0, colspan=6, sticky=W)
526     # Create the widgets
527     self.J1LimLabel = Label(self.limitTestFrame, text="J1:", bg=self.colorBG2,
fg=self.colorFG)
528     self.J1LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
529     self.J2LimLabel = Label(self.limitTestFrame, text="J2:", bg=self.colorBG2,
fg=self.colorFG)
530     self.J2LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
531     self.J3LimLabel = Label(self.limitTestFrame, text="J3:", bg=self.colorBG2,
fg=self.colorFG)
532     self.J3LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
533     self.J4LimLabel = Label(self.limitTestFrame, text="J4:", bg=self.colorBG2,
fg=self.colorFG)
534     self.J4LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
535     self.J5LimLabel = Label(self.limitTestFrame, text="J5:", bg=self.colorBG2,
fg=self.colorFG)
536     self.J5LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
537     self.J6LimLabel = Label(self.limitTestFrame, text="J6:", bg=self.colorBG2,
fg=self.colorFG)
538     self.J6LimState = Label(self.limitTestFrame, text="x", bg=self.colorBG2,
fg=self.colorFG)
539
540     # Display the widgets
541     self.J1LimLabel.grid(row=1, column=0)
542     self.J1LimState.grid(row=1, column=1)
543     self.J2LimLabel.grid(row=1, column=2)
544     self.J2LimState.grid(row=1, column=3)
545     self.J3LimLabel.grid(row=1, column=4)
546     self.J3LimState.grid(row=1, column=5)
547     self.J4LimLabel.grid(row=2, column=0)
548     self.J4LimState.grid(row=2, column=1)
549     self.J5LimLabel.grid(row=2, column=2)

```

```

550         self.J5LimState.grid(row=2, column=3)
551         self.J6LimLabel.grid(row=2, column=4)
552         self.J6LimState.grid(row=2, column=5)
553
554         # Encoder test
555         self.encoderTestFrame = Frame(self.armTestsFrame, bg=self.colorBG2)
556         self.encoderTestFrame.grid(row=2, column=0, padx=5, pady=5, sticky=W)
557         self.encoderTestButton = Button(self.encoderTestFrame, text="Test Encoders",
command=self.armController.toggleEncoderTest, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
558         self.encoderTestButton.grid(row=0, column=0, columnspan=6, sticky=W)
559         # Create the widgets
560         self.J1EncLabel = Label(self.encoderTestFrame, text="J1:", bg=self.colorBG2,
fg=self.colorFG)
561         self.J1EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
562         self.J2EncLabel = Label(self.encoderTestFrame, text="J2:", bg=self.colorBG2,
fg=self.colorFG)
563         self.J2EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
564         self.J3EncLabel = Label(self.encoderTestFrame, text="J3:", bg=self.colorBG2,
fg=self.colorFG)
565         self.J3EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
566         self.J4EncLabel = Label(self.encoderTestFrame, text="J4:", bg=self.colorBG2,
fg=self.colorFG)
567         self.J4EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
568         self.J5EncLabel = Label(self.encoderTestFrame, text="J5:", bg=self.colorBG2,
fg=self.colorFG)
569         self.J5EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
570         self.J6EncLabel = Label(self.encoderTestFrame, text="J6:", bg=self.colorBG2,
fg=self.colorFG)
571         self.J6EncState = Label(self.encoderTestFrame, text="xxxx", bg=self.colorBG2,
fg=self.colorFG)
572
573         # Display the widgets
574         self.J1EncLabel.grid(row=1, column=0)
575         self.J1EncState.grid(row=1, column=1)
576         self.J2EncLabel.grid(row=1, column=2)
577         self.J2EncState.grid(row=1, column=3)
578         self.J3EncLabel.grid(row=1, column=4)
579         self.J3EncState.grid(row=1, column=5)
580         self.J4EncLabel.grid(row=2, column=0)
581         self.J4EncState.grid(row=2, column=1)
582         self.J5EncLabel.grid(row=2, column=2)
583         self.J5EncState.grid(row=2, column=3)
584         self.J6EncLabel.grid(row=2, column=4)
585         self.J6EncState.grid(row=2, column=5)
586
587         # =====| Movement Frame |=====

```

```

588         self.moveFrame = Frame(self.armTab, bg=self.colorBG2)
589         self.moveFrame.grid(row=1, column=2, padx=5, pady=5, sticky=W+E+N+S)
590         # ===| XYZ Move |===
591         self.linearMoveFrame = Frame(self.moveFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
592         self.linearMoveFrame.grid(row=0, column=0, padx=5, pady=5, sticky=W+E+N+S)
593         self.linearMoveLabel = Label(self.linearMoveFrame, text="XYZ Move:",
bg=self.colorBG2, fg=self.colorFG)
594         self.linearMoveLabel.grid(row=0, column=0, columnspan=2, padx=5, pady=5, sticky=W)
595         # Populate xyz button
596         self.getXYZButton = Button(self.linearMoveFrame, text="Get XYZ", command =
self.armController.populateMJ, bg=self.colorAccent, fg=self.colorBG, font=("TkDefaultFont",
10, "bold"))
597         self.getXYZButton.grid(row=0, column = 2, columnspan=2, padx=5, pady=5)
598         # Send command button
599         self.linearMoveButton = Button(self.linearMoveFrame, text="Send MJ",
command=self.armController.prepMJCommand, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
600         self.linearMoveButton.grid(row=0, column=4, columnspan=2, padx=5, pady=5, sticky=E)
601         self.linearMoveButton = Button(self.linearMoveFrame, text="Send ML",
command=self.armController.prepMLCommand, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
602         self.linearMoveButton.grid(row=0, column=6, columnspan=2, padx=5, pady=5, sticky=E)
603
604         # Coordinate labels and text boxes
605         # Create them
606         self.xCoordLabel = Label(self.linearMoveFrame, text="X:", bg=self.colorBG2,
fg=self.colorFG)
607         self.xCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
608         self.yCoordLabel = Label(self.linearMoveFrame, text="Y:", bg=self.colorBG2,
fg=self.colorFG)
609         self.yCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
610         self.zCoordLabel = Label(self.linearMoveFrame, text="Z:", bg=self.colorBG2,
fg=self.colorFG)
611         self.zCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
612         self.J7CoordLabel = Label(self.linearMoveFrame, text="J7:", bg=self.colorBG2,
fg=self.colorFG)
613         self.J7CoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
614         self.RxCoordLabel = Label(self.linearMoveFrame, text="Rx:", bg=self.colorBG2,
fg=self.colorFG)
615         self.RxCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
616         self.RyCoordLabel = Label(self.linearMoveFrame, text="Ry:", bg=self.colorBG2,
fg=self.colorFG)
617         self.RyCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
618         self.RzCoordLabel = Label(self.linearMoveFrame, text="Rz:", bg=self.colorBG2,
fg=self.colorFG)

```

```

619         self.RzCoordEntry = Entry(self.linearMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
620         # Display them
621         self.xCoordLabel.grid(row=1, column=0, padx=(5, 0), pady=5)
622         self.xCoordEntry.grid(row=1, column=1, padx=(0, 5), pady=5)
623         self.yCoordLabel.grid(row=1, column=2, padx=(5, 0), pady=5)
624         self.yCoordEntry.grid(row=1, column=3, padx=(0, 5), pady=5)
625         self.zCoordLabel.grid(row=1, column=4, padx=(5, 0), pady=5)
626         self.zCoordEntry.grid(row=1, column=5, padx=(0, 5), pady=5)
627         self.J7CoordLabel.grid(row=1, column=6, padx=(0, 5), pady=5)
628         self.J7CoordEntry.grid(row=1, column=7, padx=(0, 5), pady=5)
629         self.RxCoordLabel.grid(row=2, column=4, padx=(5, 0), pady=5)
630         self.RxCoordEntry.grid(row=2, column=5, padx=(0, 5), pady=5)
631         self.RyCoordLabel.grid(row=2, column=2, padx=(5, 0), pady=5)
632         self.RyCoordEntry.grid(row=2, column=3, padx=(0, 5), pady=5)
633         self.RzCoordLabel.grid(row=2, column=0, padx=(5, 0), pady=5)
634         self.RzCoordEntry.grid(row=2, column=1, padx=(0, 5), pady=5)
635
636         # ==| Joint move |==
637         self.jointMoveFrame = Frame(self.moveFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
638         self.jointMoveFrame.grid(row=1, column=0, padx=5, pady=5, sticky=W+E+N+S)
639         self.jointMoveLabel = Label(self.jointMoveFrame, text="Joint Move:",
bg=self.colorBG2, fg=self.colorFG)
640         self.jointMoveLabel.grid(row=0, column=0, columnspan=2, padx=5, pady=5, sticky=W)
641         # Get Joints
642         self.getJointsButton = Button(self.jointMoveFrame, text="Get Joints", command =
self.armController.populateJoints, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
643         self.getJointsButton.grid(row=0, column = 2, columnspan=2, padx=5, pady=5)
644         # Send command button
645         self.jointMoveButton = Button(self.jointMoveFrame, text="Send RJ",
command=self.armController.prepRJCommand, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
646         self.jointMoveButton.grid(row=0, column=4, columnspan=3, padx=5, pady=5)
647         # Joint labels and text boxes
648         # Create them
649         self.J1CoordLabel = Label(self.jointMoveFrame, text="J1:", bg=self.colorBG2,
fg=self.colorFG)
650         self.J1CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
651         self.J2CoordLabel = Label(self.jointMoveFrame, text="J2:", bg=self.colorBG2,
fg=self.colorFG)
652         self.J2CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
653         self.J3CoordLabel = Label(self.jointMoveFrame, text="J3:", bg=self.colorBG2,
fg=self.colorFG)
654         self.J3CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
655         self.J4CoordLabel = Label(self.jointMoveFrame, text="J4:", bg=self.colorBG2,
fg=self.colorFG)

```

```

656         self.J4CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
657         self.J5CoordLabel = Label(self.jointMoveFrame, text="J5:", bg=self.colorBG2,
fg=self.colorFG)
658         self.J5CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
659         self.J6CoordLabel = Label(self.jointMoveFrame, text="J6:", bg=self.colorBG2,
fg=self.colorFG)
660         self.J6CoordEntry = Entry(self.jointMoveFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
661         # Display them
662         self.J1CoordLabel.grid(row=1, column=0, padx=(5, 0), pady=5)
663         self.J1CoordEntry.grid(row=1, column=1, padx=(0, 5), pady=5)
664         self.J2CoordLabel.grid(row=1, column=2, padx=(5, 0), pady=5)
665         self.J2CoordEntry.grid(row=1, column=3, padx=(0, 5), pady=5)
666         self.J3CoordLabel.grid(row=1, column=4, padx=(5, 0), pady=5)
667         self.J3CoordEntry.grid(row=1, column=5, padx=(0, 5), pady=5)
668         self.J4CoordLabel.grid(row=2, column=0, padx=(5, 0), pady=5)
669         self.J4CoordEntry.grid(row=2, column=1, padx=(0, 5), pady=5)
670         self.J5CoordLabel.grid(row=2, column=2, padx=(5, 0), pady=5)
671         self.J5CoordEntry.grid(row=2, column=3, padx=(0, 5), pady=5)
672         self.J6CoordLabel.grid(row=2, column=4, padx=(5, 0), pady=5)
673         self.J6CoordEntry.grid(row=2, column=5, padx=(0, 5), pady=5)
674         #Move to safe position button
675         self.moveToSafeButton = Button(self.moveFrame, text="Move to Safe Position",
command=self.armController.prepMoveSafe, width=20, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
676         self.moveToSafeButton.grid(row=6, column=0, columnspan=6, padx=5, pady=5)
677         self.moveToHomeButton = Button(self.moveFrame, text="Move to Home Position",
command=self.armController.prepMoveHome, width=20, bg=self.colorAccent, fg=self.colorBG,
font=("TkDefaultFont", 10, "bold"))
678         self.moveToHomeButton.grid(row=7, column=0, columnspan=6, padx=5, pady=5)
679         # =====| Loop Frame |=====
680         self.loopFrame = Frame(self.armTab, bg=self.colorBG2)
681         self.loopFrame.grid(row=0, column=3, padx=5, pady=5, sticky=W+E+N+S)
682         self.loopFrameLabel = Label(self.loopFrame, text="Loop Mode:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
683         self.loopFrameLabel.grid(row=0, column=0, columnspan=2, padx=5, pady=5, sticky=W)
684         self.openLoopButton = Button(self.loopFrame, text="Open Loop",
command=self.armController.setOpenLoop, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
685         self.openLoopButton.grid(row=1, column=0, padx=5, pady=5)
686         self.closedLoopButton = Button(self.loopFrame, text="Closed Loop",
command=self.armController.setClosedLoop, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
687         self.closedLoopButton.grid(row=1, column=1, padx=5, pady=5)
688         self.loopStatusLabel = Label(self.loopFrame, text="Status:", bg=self.colorBG2,
fg=self.colorFG)
689         self.loopStatusLabel.grid(row=2, column=0, padx=5, pady=5, sticky=W)
690         self.loopStatus = Label(self.loopFrame, text="Unknown", bg=self.colorBG2,
fg=self.colorFG)
691         self.loopStatus.grid(row=2, column=1, padx=5, pady=5, sticky=W)

```

```

692
693     # =====| Origin Frame |=====
694     self.originFrame = Frame(self.armTab, bg=self.colorBG2)
695     self.originFrame.grid(row=1, column=3, padx=5, pady=5, sticky=W+E+N+S)
696     self.originFrameLabel = Label(self.originFrame, text="Origin:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
697     self.originFrameLabel.grid(row=0, column=0, columnspan=3, padx=5, pady=5, sticky=W)
698     # Send command button
699     self.setOrigin = Button(self.originFrame, text="Set Origin At Current Position",
command=self.armController.setOrigin, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
700     self.setOrigin.grid(row=1, column=0, columnspan=2, padx=5, pady=5, sticky=W)
701
702     # Coordinate labels and text boxes
703     # Create them
704     self.xyzOriginFrame = Frame(self.originFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
705     self.xyzOriginFrame.grid(row=2, column=0, padx=5, pady=5)
706     # Create the widgets
707     self.xCurCoordOriginLabel = Label(self.xyzOriginFrame, text="X:", bg=self.colorBG2,
fg=self.colorFG)
708     self.xCurCoordOrigin = Label(self.xyzOriginFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
709     self.yCurCoordOriginLabel = Label(self.xyzOriginFrame, text="Y:", bg=self.colorBG2,
fg=self.colorFG)
710     self.yCurCoordOrigin = Label(self.xyzOriginFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
711     self.zCurCoordOriginLabel = Label(self.xyzOriginFrame, text="Z:", bg=self.colorBG2,
fg=self.colorFG)
712     self.zCurCoordOrigin = Label(self.xyzOriginFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
713     # Display the widgets
714     self.xCurCoordOriginLabel.grid(row=0, column=0, padx=5, pady=5)
715     self.xCurCoordOrigin.grid(row=0, column=1, padx=5, pady=5)
716     self.yCurCoordOriginLabel.grid(row=0, column=2, padx=5, pady=5)
717     self.yCurCoordOrigin.grid(row=0, column=3, padx=5, pady=5)
718     self.zCurCoordOriginLabel.grid(row=0, column=4, padx=5, pady=5)
719     self.zCurCoordOrigin.grid(row=0, column=5, padx=5, pady=5)
720     #Move to Origin
721     self.moveToOrigin = Button(self.originFrame, text="Move To Origin",
command=self.armController.moveToOrigin, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
722     self.moveToOrigin.grid(row=3, column=0, columnspan=2, padx=5, pady=5, sticky=W)
723     self.moveToRecommendedOrigin = Button(self.originFrame, text="Move To Default
Origin", command=self.armController.moveToRecommendedOrigin, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
724     self.moveToRecommendedOrigin.grid(row=4, column=0, columnspan=2, padx=5, pady=5,
sticky=W)
725
726     #Delta Origin
727     self.deltaOriginLabel = Label(self.originFrame, text="Delta from Origin:",
bg=self.colorBG2, fg=self.colorFG)

```

```

728         self.deltaOriginLabel.grid(row=5, column=0, columnspan=6, padx=5, pady=5, sticky=W)
729
730         #Delta coordinates
731         self.originDeltaFrame = Frame(self.originFrame, highlightthickness=1,
highlightbackground=self.colorFG, bg=self.colorBG2)
732         self.originDeltaFrame.grid(row=6, column=0, padx=5, pady=5)
733
734         self.xDeltaOriginLabel = Label(self.originDeltaFrame, text="ΔX:", bg=self.colorBG2,
fg=self.colorFG)
735         self.xDeltaOrigin = Label(self.originDeltaFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
736         self.yDeltaOriginLabel = Label(self.originDeltaFrame, text="ΔY:", bg=self.colorBG2,
fg=self.colorFG)
737         self.yDeltaOrigin = Label(self.originDeltaFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
738         self.zDeltaOriginLabel = Label(self.originDeltaFrame, text="ΔZ:", bg=self.colorBG2,
fg=self.colorFG)
739         self.zDeltaOrigin = Label(self.originDeltaFrame, text="xxx", bg=self.colorBG2,
fg=self.colorFG) # 'xxx' until value reported
740         self.xDeltaOriginLabel.grid(row=3, column=0, padx=5, pady=5)
741         self.xDeltaOrigin.grid(row=3, column=1, padx=5, pady=5)
742         self.yDeltaOriginLabel.grid(row=3, column=2, padx=5, pady=5)
743         self.yDeltaOrigin.grid(row=3, column=3, padx=5, pady=5)
744         self.zDeltaOriginLabel.grid(row=3, column=4, padx=5, pady=5)
745         self.zDeltaOrigin.grid(row=3, column=5, padx=5, pady=5)
746
747         def fillToolTab(self):
748             # The tool jog frame is buggy and not currently useful so it has been commented out
749             # Still here in case we want to use it in the future
750             '''
751             # ===Tool jog frame===
752             self.toolJogFrame = Frame(self.toolTab, highlightthickness=2,
highlightbackground="#000000")
753             self.toolJogFrame.grid(row=0, column=0, padx=5, pady=5, sticky=W+N+E+S)
754             # Label
755             self.toolJogLabel = Label(self.toolJogFrame, text="Tool Jog:")
756             self.toolJogLabel.grid(row=0, column=0, columnspan=3, padx=5, pady=5, sticky=W)
757             #Start and stop
758             self.startToolJog = Button(self.toolJogFrame, text="Start tool jog",
command=self.armController.startToolJog)
759             self.startToolJog.grid(row=1, column=0, padx=5, pady=5,sticky=W)
760             self.stopToolJog = Button(self.toolJogFrame, text="Stop tool jog",
command=self.armController.stopToolJog)
761             self.stopToolJog.grid(row=2, column=0, padx=5, pady=5,sticky=W)
762             self.toolJogDirection = Label(self.toolJogFrame, text="Direction: N/A")
763             self.toolJogDirection.grid(row=1,column=1)
764             self.changeToolJogDirection = Button(self.toolJogFrame, text="Change
direction",command=self.armController.changeToolJogDirection)
765             self.changeToolJogDirection.grid(row=2,column=1)
766             #Select Axis
767             self.selectAxisFrame = Frame(self.toolJogFrame)

```



```

768         self.selectAxisFrame.grid(row=3, column=0, padx=5, pady=5, sticky=W+E)
769         self.selectAxisLabel = Label(self.selectAxisFrame, text="Select Axis For Jog")
770         self.selectAxisLabel.grid(row=0, column=0, columnspan=3, padx=5, pady=5, sticky=W)
771
772         # Make the select buttons
773         self.toolJogSetX = Button(self.selectAxisFrame, text="X", command=lambda:
self.armController.selectToolJogAxis(1), width=7)
774         self.toolJogSetY = Button(self.selectAxisFrame, text="Y", command=lambda:
self.armController.selectToolJogAxis(2), width=7)
775         self.toolJogSetZ = Button(self.selectAxisFrame, text="Z", command=lambda:
self.armController.selectToolJogAxis(3), width=7)
776         self.toolJogSetRz = Button(self.selectAxisFrame, text="Rz", command=lambda:
self.armController.selectToolJogAxis(4), width=7)
777         self.toolJogSetRy = Button(self.selectAxisFrame, text="Ry", command=lambda:
self.armController.selectToolJogAxis(5), width=7)
778         self.toolJogSetRx = Button(self.selectAxisFrame, text="Rx", command=lambda:
self.armController.selectToolJogAxis(6), width=7)
779
780         # Place buttons
781         self.toolJogSetX.grid(row=1, column=0, padx=5, pady=5,)
782         self.toolJogSetY.grid(row=1, column=1, padx=5, pady=5)
783         self.toolJogSetZ.grid(row=1, column=2, padx=5, pady=5)
784         self.toolJogSetRz.grid(row=2, column=0, padx=5, pady=5)
785         self.toolJogSetRy.grid(row=2, column=1, padx=5, pady=5)
786         self.toolJogSetRx.grid(row=2, column=2, padx=5, pady=5)'''
787         # ===== Extruder Frame =====
788         self.extruderFrame = Frame(self.toolTab, bg=self.colorBG2)
789         self.extruderFrame.grid(row=0, column=1, padx=5, pady=5, sticky=W+N+E+S)
790
791         self.extruderLabel = Label(self.extruderFrame, text="Extruder Control For Testing",
font=("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
792         self.extruderLabel.grid(row=0, column=0, columnspan=2)
793         self.J7CoordLabel2 = Label(self.extruderFrame, text="J7:", bg=self.colorBG2,
fg=self.colorFG)
794         self.J7CoordEntry2 = Entry(self.extruderFrame, width=6, bg=self.colorBG2,
fg=self.colorFG)
795         self.J7CoordLabel2.grid(row=1, column=0, padx=(0, 5), pady=5)
796         self.J7CoordEntry2.grid(row=1, column=1, padx=(0, 5), pady=5)
797         self.extrudeButton = Button(self.extruderFrame, text = "Extrude",
command=self.armController.extrudeButton, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
798         self.extrudeButton.grid(row=2, column=0, padx=(0, 5), pady=5)
799         self.zeroJ7Button = Button(self.extruderFrame, text = "Zero",
command=self.armController.zeroJ7, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
800         self.zeroJ7Button.grid(row=2, column=1, padx=(0, 5), pady=5)
801
802         self.loadButton = Button(self.extruderFrame, text = "Load",
command=self.armController.loadFilament, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
803         self.loadButton.grid(row=3, column=0, padx=(0, 5), pady=5)

```

```

804         self.unloadButton = Button(self.extruderFrame, text = "Unload",
command=self.armController.unloadFilament, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
805         self.unloadButton.grid(row=3, column=1, padx=(0, 5), pady=5)
806         self.currentJ72 = Label(self.extruderFrame, text="Extruded: 0 mm", bg=self.colorBG2,
fg=self.colorFG)
807         self.currentJ72.grid(row=6, column=0, padx=5, pady=5, sticky=N+S)
808
809
810     def fillDebugTab(self):
811         # =====| Variables Frame |=====
812         self.debugVarFrame = Frame(self.debugTab, width=300, height=620, bg=self.colorBG2)
813         self.debugVarFrame.grid(row=0, column=0, padx=5, pady=5)
814         self.debugVarFrame.grid_propagate(False)
815         # ==| SerialController Variables |==
816         self.serDebugFrame = Frame(self.debugVarFrame, highlightthickness=1,
highlightbackground="#000000", bg=self.colorBG2)
817         self.serDebugFrame.grid(row=0, column=0, padx=5, pady=5, sticky=W)
818         self.serDebugLabel = Label(self.serDebugFrame, text="SerialController:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
819         self.serDebugLabel.grid(row=0, column=0, padx=5, pady=5, sticky=W)
820         # boardConnected
821         self.serDebugBoardLabel = Label(self.serDebugFrame, text="boardConnected = ",
bg=self.colorBG2, fg=self.colorFG)
822         self.serDebugBoardLabel.grid(row=1, column=0, padx=5, pady=5, sticky=W)
823         # responseReady
824         self.serDebugRespLabel = Label(self.serDebugFrame, text="responseReady = ",
bg=self.colorBG2, fg=self.colorFG)
825         self.serDebugRespLabel.grid(row=3, column=0, padx=5, pady=5, sticky=W)
826         #Override calibration button
827         self.printQueueButton = Button(self.serDebugFrame, text="Print Serial Queue",
command=self.serialController.printQueue, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
828         self.printQueueButton.grid(row=4, column=0, columnspan=2, padx=5, pady=5, sticky=W)
829
830         # ==| ArmController Variables |==
831         self.armDebugFrame = Frame(self.debugVarFrame, highlightthickness=1,
highlightbackground="#000000", bg=self.colorBG2)
832         self.armDebugFrame.grid(row=1, column=0, padx=5, pady=5, sticky=W)
833         self.armDebugLabel = Label(self.armDebugFrame, text="ArmController:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
834         self.armDebugLabel.grid(row=0, column=0, padx=5, pady=5, sticky=W)
835         # armCalibrated
836         self.armDebugCalLabel = Label(self.armDebugFrame, text="armCalibrated = ",
bg=self.colorBG2, fg=self.colorFG)
837         self.armDebugCalLabel.grid(row=1, column=0, padx=5, pady=5, sticky=W)
838         # calibrationInProgress
839         self.armDebugCalInProgLabel = Label(self.armDebugFrame, text="calibrationInProgress =
", bg=self.colorBG2, fg=self.colorFG)
840         self.armDebugCalInProgLabel.grid(row=2, column=0, padx=5, pady=5, sticky=W)
841         # calibrationState

```

```

842         self.armDebugCalStateLabel = Label(self.armDebugFrame, text="calibrationState = ",
bg=self.colorBG2, fg=self.colorFG)
843         self.armDebugCalStateLabel.grid(row=3, column=0, padx=5, pady=5, sticky=W)
844         #Override calibration button
845         self.overrideCalibrationButton = Button(self.debugVarFrame, text="Override
Calibration", command=self.armController.overrideCalibration, bg=self.colorAccent,
fg=self.colorBG, font=("TkDefaultFont", 10, "bold"))
846         self.overrideCalibrationButton.grid(row=3, column=0, columnspan=2, padx=5, pady=5,
sticky=W)
847
848         # ==| PrintController Variables |==
849         self.printDebugFrame = Frame(self.debugVarFrame, highlightthickness=1,
highlightbackground="#000000", bg=self.colorBG2)
850         self.printDebugFrame.grid(row=4, column=0, padx=5, pady=5, sticky=W)
851         self.printDebugLabel = Label(self.printDebugFrame, text="PrintController:", font=
("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
852         self.printDebugLabel.grid(row=0, column=0, padx=5, pady=5, sticky=W)
853         # armCalibrated
854         self.printDebugPrintLabel = Label(self.printDebugFrame, text="printing = ",
bg=self.colorBG2, fg=self.colorFG)
855         self.printDebugPrintLabel.grid(row=1, column=0, padx=5, pady=5, sticky=W)
856
857         # =====| Terminal Frame |=====
858
859         self.termFrame = Frame(self.debugTab, bg=self.colorBG2, highlightthickness=2,
highlightbackground="#000000", width=900)
860         self.termFrame.grid(row=0, column=1, padx=5, pady=5, sticky=W+E+N+S)
861
862         self.termVertScroll = Scrollbar(self.termFrame, orient="vertical")
863         self.termHorzScroll = Scrollbar(self.termFrame, orient="horizontal")
864
865         self.terminal = Text(self.termFrame,
866                             wrap=NONE,
867                             width=109,
868                             height=37,
869                             yscrollcommand=self.termVertScroll.set,
870                             xscrollcommand=self.termHorzScroll.set,
871                             state="disabled",
872                             background=self.colorBG2,
873                             foreground=self.colorFG
874                             )
875
876         self.termVertScroll.pack(side=RIGHT, fill=Y)
877         self.termVertScroll.config(command=self.terminal.yview)
878
879         self.termHorzScroll.pack(side=BOTTOM, fill=X)
880         self.termHorzScroll.config(command=self.terminal.xview)
881         #self.terminal.grid(row=1, column=0, columnspan=5, padx=5, pady=5)
882         self.terminal.pack(fill=BOTH)
883
884         def fillSettingsTab(self):

```

```

885 self.settingsFrame = Frame(self.settingsTab, bg=self.colorBG2)
886 self.settingsFrame.pack(side="left", fill="y", padx=10,pady=5)#grid(row=0,column=0)
887 self.settingsFrame2 = Frame(self.settingsTab, bg=self.colorBG2)
888 self.settingsFrame2.pack(side="left", fill="y", padx=10,pady=5)#grid(row=0,column=0)
889 maxSettingsPerColumn = 13
890
891 if len(self.settingsDict) <= maxSettingsPerColumn:
892     self.settingsLength1 = len(self.settingsDict)
893     self.settingsLength2 = 0
894 else:
895     self.settingsLength1 = maxSettingsPerColumn
896     self.settingsLength2 = len(self.settingsDict) - maxSettingsPerColumn
897
898 # Headers for all settings
899 header1 = Label(self.settingsFrame, text="Setting",padx=5, font=("TkDefaultFont",
900 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
901 header2 = Label(self.settingsFrame, text="Current",padx=5, font=("TkDefaultFont",
902 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
903 header3 = Label(self.settingsFrame, text="Change To",padx=5, font=("TkDefaultFont",
904 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
905 horizontalLine1 = ttk.Separator(self.settingsFrame,orient="horizontal")
906 verticalLine1 = ttk.Separator(self.settingsFrame,orient="vertical")
907 verticalLine2 = ttk.Separator(self.settingsFrame,orient="vertical")
908 horizontalLine2 = ttk.Separator(self.settingsFrame,orient="horizontal")
909 header1.grid(row=0, column=0,pady=(5,0))
910 header2.grid(row=0, column=2,pady=(5,0))
911 header3.grid(row=0, column=4,pady=(5,0))
912 horizontalLine1.grid(row=1,column=0,columnspan=5,sticky=EW)
913
914 horizontalLine2.grid(row=self.settingsLength1+2,column=0,columnspan=5,pady=5,sticky=EW)
915 verticalLine1.grid(row=0,column=1,rowspan=self.settingsLength1+3,sticky=NS,pady=
916 (0,5))
917 verticalLine2.grid(row=0,column=3,rowspan=self.settingsLength1+3,sticky=NS,pady=
918 (0,5))
919
920 if self.settingsLength2 > 0:
921     # Headers for all settings
922     header1_1 = Label(self.settingsFrame2, text="Setting",padx=5, font=
923 ("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
924 header2_1 = Label(self.settingsFrame2, text="Current",padx=5, font=
925 ("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
926 header3_1 = Label(self.settingsFrame2, text="Change To",padx=5, font=
927 ("TkDefaultFont", 12, "bold"), bg=self.colorBG2, fg=self.colorFG)
928 horizontalLine1_1 = ttk.Separator(self.settingsFrame2,orient="horizontal")
929 verticalLine1_1 = ttk.Separator(self.settingsFrame2,orient="vertical")
930 verticalLine2_1 = ttk.Separator(self.settingsFrame2,orient="vertical")
931 horizontalLine2_1 = ttk.Separator(self.settingsFrame2,orient="horizontal")
932 header1_1.grid(row=0, column=0,pady=(5,0))
933 header2_1.grid(row=0, column=2,pady=(5,0))
934 header3_1.grid(row=0, column=4,pady=(5,0))
935 horizontalLine1_1.grid(row=1,column=0,columnspan=5,sticky=EW)

```

```

927 horizontalLine2_1.grid(row=self.settingsLength2+2,column=0,columnspan=5,pady=5,sticky=EW)
928
929 verticalLine1_1.grid(row=0,column=1,rowspan=self.settingsLength2+3,sticky=NS,pady=(0,5))
930
931 verticalLine2_1.grid(row=0,column=3,rowspan=self.settingsLength2+3,sticky=NS,pady=(0,5))
932
933 #Stores the label and entry objects
934 self.entries = {}
935 self.currents = {}
936 PC = self.printController
937 AC = self.armController
938 SC = self.serialController
939
940 row = 2 #start below headers
941 for item in self.settingsDict:
942
943     attr, object = self.settingsDict[item]
944     custom = False
945     match object:
946         case "PC":
947             object = PC
948         case "AC":
949             object = AC
950         case "SC":
951             object = SC
952         case "self":
953             object = self
954         case "custom":
955             custom = True
956     if not custom:
957         currentValue = getattr(object,attr)
958     else:
959         currentValue = 0
960     if row < maxSettingsPerColumn + 2: #if there is still room in the first column
961         settingLabel = Label(self.settingsFrame, text=item, bg=self.colorBG2,
962 fg=self.colorFG)
963         self.currents[item] = Label(self.settingsFrame, text=str(currentValue),
964 bg=self.colorBG2, fg=self.colorFG)
965         self.entries[item] = Entry(self.settingsFrame,width=10, bg=self.colorBG2,
966 fg=self.colorFG)
967         #Place
968         settingLabel.grid(row=row, column=0)
969         self.currents[item].grid(row=row, column=2)
970         self.entries[item].grid(row=row, column=4,padx=5,pady=5)
971     else:
972         settingLabel = Label(self.settingsFrame2, text=item, bg=self.colorBG2,
973 fg=self.colorFG)
974         self.currents[item] = Label(self.settingsFrame2, text=str(currentValue),
975 bg=self.colorBG2, fg=self.colorFG)

```

```

969         self.entries[item] = Entry(self.settingsFrame2,width=10, bg=self.colorBG2,
fg=self.colorFG)
970
971         #Place and adjust row number for second column
972         settingLabel.grid(row=row-maxSettingsPerColumn, column=0)
973         self.currents[item].grid(row=row-maxSettingsPerColumn, column=2)
974         self.entries[item].grid(row=row-maxSettingsPerColumn,
column=4,padx=5,pady=5)
975         row += 1
976
977
978         self.setAllSettingsButton = Button(self.settingsFrame,text="Set All
Settings",command=self.setAllSettings, bg=self.colorAccent, fg=self.colorBG, font=
("TkDefaultFont", 10, "bold"))
979         self.setAllSettingsButton.grid(row=maxSettingsPerColumn-
+4,column=0,columnspan=5,padx=5,pady=5) #Settings will always be at the bottom
980
981     #endregion Tabs
982     #Set settings
983     def setAllSettings(self):
984
985         PC = self.printController
986         AC = self.armController
987         SC = self.serialController
988
989         for item in self.settingsDict:
990             valueToSet = self.entries[item].get().strip()
991
992             attr, object = self.settingsDict[item]
993             #Custom attribute to send a command to change the teensy's backlash variable
994             if object == "custom":
995                 match attr:
996                     case "J5BacklashFix":
997                         command = f"BFA{float(valueToSet)*43.720}\n"
998                         self.serialController.sendSerial(command)
999                 continue
1000             match object:
1001                 case "PC":
1002                     object = PC
1003                 case "AC":
1004                     object = AC
1005                 case "SC":
1006                     object = SC
1007                 case "self":
1008                     object = self
1009
1010             currentValue = getattr(object,attr)
1011             #If there is a value to set
1012             if valueToSet != "":
1013                 try:
1014                     # Convert to the same type as the existing value

```

```

1015         correctType = type(currentValue)
1016         #Special case to handle bool because any non empty string is converted to
True
1017         if correctType == bool:
1018             #Option for several true cases and false cases
1019             if valueToSet.lower() in ("true", "1", "yes", "y"):
1020                 valueToSet = True
1021             elif valueToSet.lower() in ("false", "0", "no", "n"):
1022                 valueToSet = False
1023             else:
1024                 raise ValueError
1025         if correctType != None:
1026             valueToSet = correctType(valueToSet)
1027         setattr(object, attr, valueToSet)
1028
1029     except ValueError:
1030         self.terminalPrint(f"Invalid value for {item}")
1031
1032     #Get current value to make sure the value was actually set and display
1033     #Also update all values even if unchanged by user
1034     currentValue = getattr(object,attr)
1035     self.currents[item].config(text=str(currentValue))
1036
1037     #Recreate parameters
1038     PC.defaultPrintParameters =
MoveParameters(PC.speed,PC.acceleration,PC.decceleration,PC.ramp,PC.printOpenLoopControl,"m")
1039     PC.changeRecommendedOrigin(PC.plateHeight) #change origin with plate height
1040     AC.extruder_deg_per_mm = AC.extruder_deg_per_mm_cool * AC.heatedFilamentMultiplier
1041
1042     #region main update function
1043     def update(self):
1044         self.updateDebugVars() # Update the debug tab variables
1045
1046         # =====| SerialController |=====
1047
1048         # =====| ArmController |=====
1049
1050         # =====| PrintController |=====
1051         #Each print loop runs in the thread so that it can "wait" and not halt the UI
1052         if self.printController.printing and not self.printThreadStarted and not
self.printController.printPaused:
1053             printThread = threading.Thread(target=self.printController.printLoop, name =
"Print Loop Thread")
1054             printThread.start()
1055             #Variable to signal when the thread finishes
1056             self.printThreadStarted = True
1057
1058         # TODO: Temporary
1059         self.temperatureController.updateTemp()
1060
1061         # =====| Home Tab |=====

```

```

1062
1063     # Thermals
1064     hotendData = self.temperatureController.prevHotendTemps
1065     bedData = self.temperatureController.prevBedTemps
1066     if ((len(hotendData) == len(bedData) > 0) and (len(hotendData) > 1)):
1067         # Update the labels
1068         self.hotendHomeActual.config(text=f"{self.temperatureControlle-
r.hotendTempCelsius:.2f}")
1069         #self.hotendHomeTarget.config(text=round(self.temperatureControlle-
r.hotendTargetTemp, 2))
1070         self.bedHomeActual.config(text=f"{self.temperatureControlle-
r.bedTempCelsius:.2f}")
1071         #self.bedHomeTarget.config(text=round(self.temperatureController.bedTargetTemp,
2))
1072
1073         # Update the plot
1074         xData = np.linspace(self.updateDelay / -1000 * len(hotendData), 0,
len(hotendData))
1075         self.thermalHotendLine.set_data(xData, hotendData)
1076         self.thermalBedLine.set_data(xData, bedData)
1077         self.thermalPlot.set_xlim(min(xData), max(xData))
1078         self.yMax = max([max(hotendData), max(bedData), self.temperatureControlle-
r.hotendTargetTemp, self.temperatureController.bedTargetTemp]) + 10
1079
1080         self.thermalPlot.set_ylim(0, self.yMax)
1081         self.hotendTargetLine.remove()
1082         self.bedTargetLine.remove()
1083         self.hotendTargetLine = self.thermalPlot.axhline(y=self.temperatureControlle-
r.hotendTargetTemp, color='r', linestyle='--')
1084         self.bedTargetLine = self.thermalPlot.axhline(y=self.temperatureControlle-
r.bedTargetTemp, color='b', linestyle='--')
1085         self.thermalCanvas.draw()
1086
1087     # Arm Info
1088     if self.serialController.boardConnected:
1089         self.connectedStatusHome.config(text="Arm Connected", fg="#00FF00")
1090     else:
1091         self.connectedStatusHome.config(text="Arm Disconnected", fg="#FF0000")
1092     if self.armController.armCalibrated:
1093         self.calibrationStatusHome.config(text="Arm Calibrated", fg="#00FF00")
1094     else:
1095         self.calibrationStatusHome.config(text="Arm NOT Calibrated", fg="#FF0000")
1096
1097     # Set up another call to the update function after updateDelay milliseconds
1098     self.after(self.updateDelay, self.update)
1099
1100 #endregion main update function
1101 #region other functions
1102 def updateDebugVars(self):
1103     # SerialController vars

```



```

1104         self.serDebugBoardLabel.config(text=f"boardConnected =
{self.serialController.boardConnected}")
1105         self.serDebugRespLabel.config(text=f"responseReady =
{self.serialController.responseReady}")
1106
1107         # ArmController vars
1108         self.armDebugCalLabel.config(text=f"armCalibrated =
{self.armController.armCalibrated}")
1109         self.armDebugCalInProgLabel.config(text=f"calibrationInProgress =
{self.armController.calibrationInProgress}")
1110         self.armDebugCalStateLabel.config(text=f"calibrationState =
{self.armController.calibrationState}")
1111
1112
1113         # PrintController vars
1114         self.printDebugPrintLabel.config(text=f"printing = {self.printController.printing}")
1115
1116         # Called whenever the selection in the port dropdown is changed
1117         def portSelectionChanged(self, *args):
1118             print("Port Selection Changed")
1119             # Check if the selected port is the default "Select Port" text
1120             if self.portSelection.get() == "Select Port":
1121                 # If so, disable the connect button to avoid connecting to nothing
1122                 self.connectButton.config(state="disabled")
1123             else:
1124                 # If not, enable the connect button
1125                 self.connectButton.config(state="normal")
1126
1127         def clearEntryFocus(self, event):
1128             # Check if our current focus is a specific entry widget and if the new focus is
different
1129             # If so we unfocus the entry and do some other stuff if needed
1130             if self.root.focus_get() == self.hotendHomeTarget and event.widget !=
self.hotendHomeTarget:
1131                 self.temperatureControlle-
r.setHotendTargetTemp(float(self.hotendHomeTarget.get()), False)
1132                 self.root.focus_set()
1133             elif self.root.focus_get() == self.bedHomeTarget and event.widget !=
self.bedHomeTarget:
1134                 self.temperatureController.setBedTargetTemp(float(self.bedHomeTarget.get()),
False)
1135                 self.root.focus_set()
1136
1137         def windowDimensions(self, event):
1138             # Used for debugging purposes but it simply prints the window width and height
1139             print(f"{self.winfo_width()}x{self.winfo_height()}")
1140             #endregion other functions
1141
1142
1143         #region Print Functions
1144         # Used to print to the in window terminal

```

```

1145     def terminalPrint(self, message):
1146         print(message) # Print to the Python terminal as well
1147         self.terminal.config(state="normal") # Need to enable to modify
1148         self.terminal.insert(END, f"{datetime.now().now()}| {message}\n") # Print the
message with the current time fixated at the front
1149         self.terminal.config(state="disabled") # Disable again to avoid user changes
1150         self.terminal.see("end") # Forces terminal to autoscroll with new text. Probably want
to make this a toggle option
1151
1152     # Used to print important info to the status bar
1153     def statusPrint(self, message):
1154         # TODO: Add extra check for if text is too long and do a fancy '...' or something to
show that there is more
1155         # Update the status label
1156         self.statusLabel.config(text=f"Status: {message}")
1157         # Also print the full message to the terminal
1158         self.terminalPrint(message)
1159
1160     def warningPrint(self, message):
1161         self.BlinkLED = True # Used to signal the update loop to start blinking the LED
1162         #Display a 2nd message based on ignoreflags
1163         message2 = ""
1164         if self.printController.ignoreFlags and self.printController.printing:
1165             message2 = "\nPressing ok will resume movement"
1166         elif self.printController.printing:
1167             message2 = "\nPrinting will stop"
1168         #show message
1169         messagebox.showinfo("Warning! ", message+message2)
1170         self.statusPrint(message)
1171         self.BlinkLED = False # Stop blinking the LED
1172
1173     #endregion print functions
1174     #region LED Update Loop
1175     def updateLED(self):
1176         while self.ledThreadRunning:
1177             # LED Blinking takes priority over simple on/off
1178             if self.BlinkLED:
1179                 # Check the current state of the LED
1180                 tempState = GPIO.input(self.LEDPin)
1181                 # Invert the state
1182                 if tempState == GPIO.HIGH:
1183                     GPIO.output(self.LEDPin, GPIO.LOW)
1184                 else:
1185                     GPIO.output(self.LEDPin, GPIO.HIGH)
1186             # If the LEDOn flag is set, turn the LED on
1187             elif self.LEDOn:
1188                 GPIO.output(self.LEDPin, GPIO.HIGH)
1189             # If no flags are set, turn the LED off
1190             else:
1191                 GPIO.output(self.LEDPin, GPIO.LOW)
1192             # Use this to set blinking rate and how quickly the LED updates

```

```

1193         time.sleep(0.25)
1194
1195     #endregion
1196     #region popup
1197     def createPostCalibration(self):
1198         # Create a new top-level window
1199         popup = Toplevel(self.root)
1200         popup.wm_title("Post Calibration")
1201         ws = self.wininfo_screenwidth() # Get screen width
1202         hs = self.wininfo_screenheight() # Get screen height
1203         x = int((ws/2) - (200/2)) # Calculate x position for window to be in the center of
the screen
1204         y = int((hs/2) - (200/2)) # Calculate y position for window to be in the center of
the screen
1205         popup.geometry(f"350x300+{x}+{y}")
1206         # Ensure the popup stays on top of the main window
1207         popup.grab_set()
1208         popup.lift()
1209         # Add widgets to the popup
1210         label2 = Label(popup, text="Adjust calibration without limit switches.\n Works
accumalatively. \n Be careful to not move past limit switches.")
1211         label2.grid(row=1,column=0)
1212         self.calOffsetFrameP = Frame(popup)
1213         self.calOffsetFrameP.grid(row=2, column=0, padx=5, pady=5, sticky=W+E)
1214         self.offsetLabel = Label(self.calOffsetFrameP, text="Joint Offsets:")
1215         self.offsetLabel.grid(row=0, column=0, columnspan=5, padx=5, pady=5, sticky=W)
1216         # Make the widgets
1217         self.J10ffsetLabelP = Label(self.calOffsetFrameP,text="J1:")
1218         self.J10ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1219         self.J20ffsetLabelP = Label(self.calOffsetFrameP,text="J2:")
1220         self.J20ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1221         self.J30ffsetLabelP = Label(self.calOffsetFrameP,text="J3:")
1222         self.J30ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1223         self.J40ffsetLabelP = Label(self.calOffsetFrameP,text="J4:")
1224         self.J40ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1225         self.J50ffsetLabelP = Label(self.calOffsetFrameP,text="J5:")
1226         self.J50ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1227         self.J60ffsetLabelP = Label(self.calOffsetFrameP,text="J6:")
1228         self.J60ffsetEntryP = Entry(self.calOffsetFrameP, width=4)
1229         # Grid the widgets
1230         self.J10ffsetLabelP.grid(row=1, column=0, padx=5, pady=5)
1231         self.J10ffsetEntryP.grid(row=1, column=1, padx=5, pady=5)
1232         self.J20ffsetLabelP.grid(row=1, column=2, padx=5, pady=5)
1233         self.J20ffsetEntryP.grid(row=1, column=3, padx=5, pady=5)
1234         self.J30ffsetLabelP.grid(row=1, column=4, padx=5, pady=5)
1235         self.J30ffsetEntryP.grid(row=1, column=5, padx=5, pady=5)
1236         self.J40ffsetLabelP.grid(row=2, column=0, padx=5, pady=5)
1237         self.J40ffsetEntryP.grid(row=2, column=1, padx=5, pady=5)
1238         self.J50ffsetLabelP.grid(row=2, column=2, padx=5, pady=5)
1239         self.J50ffsetEntryP.grid(row=2, column=3, padx=5, pady=5)
1240         self.J60ffsetLabelP.grid(row=2, column=4, padx=5, pady=5)

```

```
1241     self.J60offsetEntryP.grid(row=2, column=5, padx=5, pady=5)
1242     # Auto fill a value of '0'
1243     self.J10offsetEntryP.insert(0, "0")
1244     self.J20offsetEntryP.insert(0, "0")
1245     self.J30offsetEntryP.insert(0, "0")
1246     self.J40offsetEntryP.insert(0, "0")
1247     self.J50offsetEntryP.insert(0, "0")
1248     self.J60offsetEntryP.insert(0, "0")
1249
1250     # Individual calibration buttons
1251     self.indivCalPFrame = Frame(popup)
1252     self.indivCalPFrame.grid(row=3, column=0, padx=5, pady=5, sticky=W+E)
1253     # Label
1254     self.indivCalPLabel = Label(self.indivCalPFrame, text="Individual Calibrations:")
1255     self.indivCalPLabel.grid(row=0, column=0, columnspan=3, padx=5, pady=5, sticky=W)
1256     # Make the buttons
1257     self.calPJ1Button = Button(self.indivCalPFrame, text="Cal J1", command=lambda:
self.armController.startPostCalibration(1, 0, 0, 0, 0, 0), width=7)
1258     self.calPJ2Button = Button(self.indivCalPFrame, text="Cal J2", command=lambda:
self.armController.startPostCalibration(0, 1, 0, 0, 0, 0), width=7)
1259     self.calPJ3Button = Button(self.indivCalPFrame, text="Cal J3", command=lambda:
self.armController.startPostCalibration(0, 0, 1, 0, 0, 0), width=7)
1260     self.calPJ4Button = Button(self.indivCalPFrame, text="Cal J4", command=lambda:
self.armController.startPostCalibration(0, 0, 0, 1, 0, 0), width=7)
1261     self.calPJ5Button = Button(self.indivCalPFrame, text="Cal J5", command=lambda:
self.armController.startPostCalibration(0, 0, 0, 0, 1, 0), width=7)
1262     self.calPJ6Button = Button(self.indivCalPFrame, text="Cal J6", command=lambda:
self.armController.startPostCalibration(0, 0, 0, 0, 0, 1), width=7)
1263     # Place buttons
1264     self.calPJ1Button.grid(row=1, column=0, padx=5, pady=5,)
1265     self.calPJ2Button.grid(row=1, column=1, padx=5, pady=5)
1266     self.calPJ3Button.grid(row=1, column=2, padx=5, pady=5)
1267     self.calPJ4Button.grid(row=2, column=0, padx=5, pady=5)
1268     self.calPJ5Button.grid(row=2, column=1, padx=5, pady=5)
1269     self.calPJ6Button.grid(row=2, column=2, padx=5, pady=5)
1270     # Add a button to close the popup
1271     close_button = Button(popup, text="Close", command=popup.destroy)
1272     close_button.pack(pady=10)
1273
1274     #endregion popup
1275
1276 #Start the Program
1277 if __name__ == "__main__":
1278     app = Tkwindow()
1279     app.mainloop()
```